In [ ]:
```
# EXPERIMENT 2 – NUMPY OPERATIONS
PIN No.: 241UDS0110

## AIM
To study and practice NumPy operations including array creation, attributes, indexing, slicing, broadcasting, arithmetic,
statistical operations, concatenation, reshaping, sorting, splitting, and retrieving selected data.

## Dataset
This experiment uses a new dataset about weekly e-learning course registrations on Android, iOS, and Windows. The dataset
values differ from the faculty PDF. Keep `Experiment_2_241UDS0110_Elearning_Registrations.csv` in the same folder as this
notebook.
```

In [ ]:
```
## Step 1 – Install and Import NumPy
```

In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
## Step 2 – Load the Dataset and Create Arrays for Three Platforms
```

In [3]:
```
registrations_df_241UDS0110 = pd.read_csv("Experiment_2_241UDS0110_Elearning_Registrations.csv")
registrations_df_241UDS0110
```

Out[3]:

	PIN	Platform	Week	Course_Registrations
0	241UDS0110	Android	Week1	735
1	241UDS0110	Android	Week2	820
2	241UDS0110	Android	Week3	790
3	241UDS0110	Android	Week4	860
4	241UDS0110	iOS	Week1	640
5	241UDS0110	iOS	Week2	710
6	241UDS0110	iOS	Week3	755
7	241UDS0110	iOS	Week4	805
8	241UDS0110	Windows	Week1	410
9	241UDS0110	Windows	Week2	465
10	241UDS0110	Windows	Week3	440
11	241UDS0110	Windows	Week4	500

In [4]:
```
android_registrations_241UDS0110 = registrations_df_241UDS0110.loc[
    registrations_df_241UDS0110["Platform"] == "Android", "Course_Registrations"
].to_numpy()

ios_registrations_241UDS0110 = registrations_df_241UDS0110.loc[
    registrations_df_241UDS0110["Platform"] == "iOS", "Course_Registrations"
].to_numpy()

windows_registrations_241UDS0110 = registrations_df_241UDS0110.loc[
    registrations_df_241UDS0110["Platform"] == "Windows", "Course_Registrations"
].to_numpy()

android_registrations_241UDS0110, ios_registrations_241UDS0110, windows_registrations_241UDS0110
```

Out[4]:
```
(array([735, 820, 790, 860]),
 array([640, 710, 755, 805]),
 array([410, 465, 440, 500]))
```

In [5]:
```
## Step 3 – Check Array Properties
```

In [6]:
```
android_registrations_241UDS0110.ndim
```

Out[6]:
```
1
```

In [7]:
```
ios_registrations_241UDS0110.shape
```

Out[7]:
```
(4,)
```

In [8]:
```
windows_registrations_241UDS0110.size
```

Out[8]:
```
4
```

In [9]:
```
android_registrations_241UDS0110.dtype
```

Out[9]:
```
dtype('int64')
```

In [10]:
```
android_registrations_241UDS0110.itemsize, ios_registrations_241UDS0110.itemsize, windows_registrations_241UDS0110.itemsize
```

Out[10]:
```
(8, 8, 8)
```

In [12]:
```
windows_registrations_241UDS0110[0]
```

Out[12]:
```
np.int64(410)
```

In [ ]:
```
## Step 4 — Retrieve Data Using Indexing and Slicing
Indexing retrieves one element. Slicing retrieves a range. `np.unique()` returns sorted unique values.
```

In [13]:
```
np.unique(ios_registrations_241UDS0110)
```

Out[13]:
```
array([640, 710, 755, 805])
```

In [14]:
```
android_registrations_241UDS0110[0:2]
```

Out[14]:
```
array([735, 820])
```

In [15]:
```
ios_registrations_241UDS0110[2:]
```

Out[15]:
```
array([755, 805])
```

In [16]:
```
windows_registrations_241UDS0110[-3:]
```

Out[16]:
```
array([465, 440, 500])
```

In [ ]:
```
## Step 5 — Assign New Values Using Broadcasting
Copy the Android array, append two new values, then update the last two values. The original dataset array remains
unchanged.
```

In [18]:
```
android_registrations_new_241UDS0110 = android_registrations_241UDS0110.copy()
android_registrations_new_241UDS0110
```

Out[18]:
```
array([735, 820, 790, 860])
```

In [19]:
```
android_registrations_new_241UDS0110 = np.append(
    android_registrations_new_241UDS0110,
    [900, 780]
)
android_registrations_new_241UDS0110
```

Out[19]:
```
array([735, 820, 790, 860, 900, 780])
```

In [20]:
```
android_registrations_new_241UDS0110[4:6] = [920, 810]
android_registrations_new_241UDS0110
```

Out[20]:
```
array([735, 820, 790, 860, 920, 810])
```

In [21]:
```
## Step 6 — Arithmetic Operations
```

In [22]:
```
windows_registrations_241UDS0110
```

Out[22]:
```
array([410, 465, 440, 500])
```

In [23]:
```
windows_registrations_241UDS0110 + 20
```

Out[23]:
```
array([430, 485, 460, 520])
```

In [24]:
```
windows_registrations_241UDS0110[3] - 10
```

Out[24]:
```
np.int64(490)
```

In [25]:
```
windows_registrations_241UDS0110[0] * 2
```

Out[25]:
```
np.int64(820)
```

In [26]:
```
windows_registrations_241UDS0110[1:3] / 2
```

Out[26]:
```
array([232.5, 220. ])
```

In [27]:
```
## Step 7 — Statistical Operations
```

In [28]:
```
android_registrations_241UDS0110, ios_registrations_241UDS0110, windows_registrations_241UDS0110
```

Out[28]:
```
(array([735, 820, 790, 860]),
 array([640, 710, 755, 805]),
 array([410, 465, 440, 500]))
```

In [29]:
```
np.sum(android_registrations_241UDS0110)
```

Out[29]:
```
np.int64(3205)
```

In [30]:
```
np.max(android_registrations_241UDS0110), np.min(android_registrations_241UDS0110)
```

Out[30]:
```
(np.int64(860), np.int64(735))
```

In [31]:
```
print(
    np.mean(android_registrations_241UDS0110),
    np.mean(ios_registrations_241UDS0110),
    np.mean(windows_registrations_241UDS0110)
)
```

```
801.25 727.5 453.75
```

In [32]:
```
## Step 8 — Combine All Platform Data
```

In [33]:
```
android_registrations_241UDS0110.ndim, ios_registrations_241UDS0110.ndim, windows_registrations_241UDS0110.ndim
```

Out[33]:
```
(1, 1, 1)
```

In [34]:
```
all_platform_registrations_241UDS0110 = np.concatenate([
    android_registrations_241UDS0110,
    ios_registrations_241UDS0110,
    windows_registrations_241UDS0110
])
all_platform_registrations_241UDS0110
```

Out[34]:
```
array([735, 820, 790, 860, 640, 710, 755, 805, 410, 465, 440, 500])
```

In [35]:
```
np.sum(all_platform_registrations_241UDS0110)
```

Out[35]:
```
np.int64(7930)
```

In [36]:
```
## Step 9 — Reshape into a 3 × 4 Matrix
```

In [37]:
```
all_platform_registrations_241UDS0110.ndim, all_platform_registrations_241UDS0110.shape
```

Out[37]:
```
(1, (12,))
```

In [38]:
```
all_platform_registrations_241UDS0110 = all_platform_registrations_241UDS0110.reshape(3, 4)
all_platform_registrations_241UDS0110
```

Out[38]:
```
array([[735, 820, 790, 860],
       [640, 710, 755, 805],
       [410, 465, 440, 500]])
```

In [39]:
```
all_platform_registrations_241UDS0110.ndim, all_platform_registrations_241UDS0110.shape
```

Out[39]:
```
(2, (3, 4))
```

In [ ]:
```
## Step 10 — Sort the Array
`axis=1` sorts values within each row; `axis=0` sorts values within each column. These expressions return sorted arrays
without modifying the original matrix.
```

In [41]:
```
all_platform_registrations_241UDS0110
```

Out[41]:
```
array([[735, 820, 790, 860],
       [640, 710, 755, 805],
       [410, 465, 440, 500]])
```

In [42]:
```
all_platform_registrations_241UDS0110[0]
```

Out[42]:
```
array([735, 820, 790, 860])
```

In [43]:
```
np.sort(all_platform_registrations_241UDS0110, axis=1)
```

Out[43]:
```
array([[735, 790, 820, 860],
       [640, 710, 755, 805],
       [410, 440, 465, 500]])
```

In [44]:
```
np.sort(all_platform_registrations_241UDS0110, axis=0)
```

Out[44]:
```
array([[410, 465, 440, 500],
       [640, 710, 755, 805],
       [735, 820, 790, 860]])
```

In [45]:
```
## Step 11 — Split the Array
```

In [46]:
```
split_android_241UDS0110, split_ios_241UDS0110, split_windows_241UDS0110 = np.split(
    all_platform_registrations_241UDS0110,
    [1, 2],
    axis=0
)
```

In [47]:
```
split_android_241UDS0110
```

Out[47]:
```
array([[735, 820, 790, 860]])
```

In [48]:
```
split_ios_241UDS0110
```

Out[48]:
```
array([[640, 710, 755, 805]])
```

In [49]:
```
split_windows_241UDS0110
```

Out[49]:
```
array([[410, 465, 440, 500]])
```

In [50]:
```
## Step 12 — Retrieve Week 2 Registrations of All Platforms
```

In [51]:
```
all_platform_registrations_241UDS0110
```

Out[51]:
```
array([[735, 820, 790, 860],
       [640, 710, 755, 805],
       [410, 465, 440, 500]])
```

In [52]:
```
week_2_registrations_241UDS0110 = all_platform_registrations_241UDS0110[:3, 1:2]
print(week_2_registrations_241UDS0110)
```

```
[[820]
 [710]
 [465]]
```

In [ ]:
```
## RESULT
Using NumPy, array creation and properties, indexing, slicing, appending and updating values, arithmetic and statistical
operations, concatenation, reshaping, sorting, splitting, and selecting Week 2 registrations were performed on the e-
learning registration dataset.
```